

Linux Firewall and Secure Remote Access

Laboratory 4

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	5 to 6 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name OBOTU ISAAC IKEH

Registration number C1126OCO17005

Submission date 30th September, 2026

Purpose and Scenario

This laboratory develops the defensive skills required to identify exposed services, enforce host firewall rules and configure SSH key authentication. Every change includes a validation and recovery step so that security controls remain observable and reversible.

You are hardening a Linux server before it joins an internal training network. The system currently exposes services without a documented policy. You must establish a baseline, permit only approved access, test the controls from a client and secure administrative access with an SSH key.

Learning Outcomes

- Identify listening TCP and UDP services and the processes that own them.
- Explain INPUT, OUTPUT and FORWARD traffic in Linux packet filtering.
- Create, verify, order and remove UFW rules safely.
- Inspect iptables or nftables compatibility rules and counters.
- Generate and deploy an Ed25519 SSH key pair.
- Validate SSH configuration before reloading the service.
- Test firewall and SSH controls from a separate authorised client.
- Document rollback steps and residual risk.

Environment and Prerequisites

- A server VM and a client VM on 192.168.56.0/24 are recommended.
- Console access to the server VM and a fresh snapshot.
- sudo permission and OpenSSH server installed on the server.
- UFW, iptables, netcat and Python 3.
- A confirmed client-to-server route before firewall changes.

Safety and Authorisation

- Do not perform firewall changes on a remote-only system without console recovery access.
- Allow and test SSH before enabling a default-deny firewall policy.
- Keep one authenticated SSH session open while testing a new SSH configuration in a second session.
- Never submit a private SSH key. Only the public key fingerprint may appear in evidence.
- Apply rules only to the authorised laboratory hosts and remove temporary services during cleanup.

Evidence Requirements

Item	Required evidence	Included
1	Baseline service and socket inventory.	[]
2	UFW policy and numbered-rule evidence before and after testing.	[]
3	Client-side allowed and denied connection tests.	[]

Item	Required evidence	Included
4	Safe iptables custom-chain exercise and cleanup.	[]
5	SSH public-key setup, fingerprint, successful login and server validation.	[]
6	Final exposure review and rollback record.	[]

Laboratory Topology

System	Example address	Purpose
Server VM	192.168.56.10	UFW, service inventory and SSH hardening
Client VM	192.168.56.20	Authorised connectivity and rule testing
Private network	192.168.56.0/24	Host-only or internal virtual network

Part 1 Baseline and Recovery Preparation

1. Take a snapshot of the server VM and confirm local console access.
2. Record interfaces, routes, listening sockets, running services and current firewall status.
3. Confirm that the client can reach the server before security rules are changed.

```
mkdir -p ~/core102_lab4 && cd ~/core102_lab4
date
hostname
ip -br addr
ip route
sudo ss -tulpn
sudo systemctl --type=service --state=running --no-pager
sudo ufw status verbose
sudo iptables -L -n -v --line-numbers
sudo ip6tables -L -n -v --line-numbers 2>/dev/null || true
```

Port and protocol	Bind address	Process	Expected purpose	Keep or remove
22 / tcp	0.0.0.0 / [::]	sshd	Remote administrative access and command execution over SSH.	Keep (Hardened with Ed25519 key authentication).

Port and protocol	Bind address	Process	Expected purpose	Keep or remove
8080 / tcp	0.0.0.0	python3 (http.server)	Temporary web service used for testing UFW rule verification and packet filtering behavior.	Remove (Stopped and decommissioned during final cleanup).
123 / udp (Optional/Example)	127.0.0.1	systemd-timesyncd	Local network time synchronization daemon.	Keep (Local system service).
631 / tcp (Optional/Example)	127.0.0.1	cupsd	Local printing and Common UNIX Printing System service.	Remove / Restrict (Not needed for headless server baseline).

```

(obotu@kali)-[~]
$ mkdir -p ~/core102_lab4 66 cd ~/core102_lab4

(obotu@kali)-[~/core102_lab4]
$ date
Wed 30 Sep 2026 12:25:22 WAT

(obotu@kali)-[~/core102_lab4]
$ hostname
kali

(obotu@kali)-[~/core102_lab4]
$ ip -br addr
lo        UNKNOWN    127.0.0.1/8 ::1/128
eth0      UP            192.168.56.101/24 fe80::a00:27ff:fe05:cd77/64
eth1      UP

```

```
File Machine View Input Devices Help
obotu@kali: ~/core102_lab4
$ ip route
192.168.56.0/24 dev eth1 proto kernel scope link src 192.168.56.101 metric 100
(obotu@kali)~[~/core102_lab4]
$ sudo ss -tuln
Netid      State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port
(obotu@kali)~[~/core102_lab4]
$ sudo systemctl --type=service --state=running --no-pager
systemctl: unrecognized option '--state=running'
(obotu@kali)~[~/core102_lab4]
$ sudo systemctl --type=service --state=running --no-pager
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
accounts-daemon.service             loaded active running Accounts Service
colord.service                       loaded active running Manage, Install and Generate Color Profiles
cron.service                        loaded active running Regular background program processing daemon
dbus.service                        loaded active running D-Bus System Message Bus
getty@tty1.service                  loaded active running Getty on tty1
haveged.service                     loaded active running Entropy Daemon based on the HAVEGE algorithm
lightdm.service                     loaded active running Light Display Manager
ModemManager.service                loaded active running Modem Manager
NetworkManager.service              loaded active running Network Manager
polkit.service                       loaded active running Authorization Manager
rtkit-daemon.service                loaded active running RealtimeKit Scheduling Policy Service
systemd-journald.service             loaded active running Journal Service
systemd-logind.service               loaded active running User Login Management
polkit.service                       loaded active running Authorization Manager
rtkit-daemon.service                loaded active running RealtimeKit Scheduling Policy Service
systemd-journald.service             loaded active running Journal Service
systemd-logind.service               loaded active running User Login Management
systemd-udevd.service                loaded active running Rule-based Manager for Device Events and Files
systemd-userdbd.service              loaded active running User Database Manager
udisks2.service                     loaded active running Disk Manager
upower.service                       loaded active running Daemon for power management
user@1000.service                   loaded active running User Manager for UID 1000
virtualbox-guest-utils.service       loaded active running Virtualbox guest utils
Legend: LOAD    → Reflects whether the unit definition was properly loaded.
ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
SUB    → The low-level unit activation state, values depend on unit type.
19 loaded units listed.
(obotu@kali)~[~/core102_lab4]
$ sudo ufw status verbose
Status: inactive
(obotu@kali)~[~/core102_lab4]
$ sudo iptables -L -n -v --line-numbers
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                destination
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
```

```
File Machine View Input Devices Help
obotu@kali: ~/core102_lab4
$ sudo ufw status verbose
Status: inactive

(obotu@kali)-[~/core102_lab4]
$ sudo iptables -L -n -v --line-numbers
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source         destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source         destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source         destination

(obotu@kali)-[~/core102_lab4]
$ sudo ip6tables -L -n -v --line-numbers 2>/dev/null || true
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source         destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source         destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source         destination

(obotu@kali)-[~/core102_lab4]
$
```

Part 2 Controlled Test Service

Start a temporary web service so firewall behaviour can be tested without installing a permanent application.

```
mkdir -p ~/core102_lab4/web
echo 'CORE102 authorised firewall test' > ~/core102_lab4/web/index.html
cd ~/core102_lab4/web
python3 -m http.server 8080 --bind 0.0.0.0
```

In a second server terminal, confirm that the process listens on TCP port 8080. From the client, test the connection before applying firewall rules.

```
# Server
sudo ss -ltnp | grep ':8080'

# Client
curl -v http://192.168.56.10:8080/
nc -vz 192.168.56.10 8080
```

```
(obotu@kali)-[~/core102_lab4]
$ mkdir -p ~/core102_lab4/web

(obotu@kali)-[~/core102_lab4]
$ echo 'CORE102 authorised firewall test' > ~/core102_lab4/web/index.html

(obotu@kali)-[~/core102_lab4]
$ cd ~/core102_lab4/web

(obotu@kali)-[~/core102_lab4/web]
$ python -m http.server 8080 --bind 0.0.0.0
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

```
File Machine View Input Devices Help
obotu@kali: ~
$ sudo ss -ltnp | grep ':8080'
[sudo] password for obotu:
LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:(("python",pid=35481,fd=3))

(obotu@kali)-[~]
$ sudo ufw status verbose
Status: inactive

(obotu@kali)-[~]
$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)

(obotu@kali)-[~]
$ sudo ufw default deny outgoing
Default outgoing policy changed to 'deny'
(be sure to update your rules accordingly)

(obotu@kali)-[~]
$ sudo ufw allow from 192.168.56.0/24 to any port 22 tcp comment 'CORE102 SSH'
ERROR: Wrong number of arguments

(obotu@kali)-[~]
$ sudo ufw default deny outgoing
Default outgoing policy changed to 'deny'
(be sure to update your rules accordingly)

(obotu@kali)-[~]
$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)

(obotu@kali)-[~]
$ sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp comment 'CORE102 SSH'
Rules updated

(obotu@kali)-[~]
$ sudo ufw allow from 192.168.56.0/24 to any port 8080 proto tcp comment 'CORE102 test web'
Rules updated

(obotu@kali)-[~]
$ sudo ufw deny 23/tcp comment 'Block Telnet'
Rules updated

(obotu@kali)-[~]
$ sudo ufw deny 23/tcp comment 'Block Telnet'
Rules updated (v6)

(obotu@kali)-[~]
$ sudo ufw deny 23/tcp comment 'Block Telnet'
Rules updated (v6)
```

```

File Machine View Input Devices Help
obotu@kali -
Session Actions Edit View Help
(obotu@kali)-[~]
$ sudo ufw enable
Firewall is active and enabled on system startup
(obotu@kali)-[~]
$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 2] 8080/tcp ALLOW IN 192.168.56.0/24 # CORE102 test web
[ 3] 23/tcp DENY IN Anywhere # Block Telnet
[ 4] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

(obotu@kali)-[~]
$ sudo ss -ltnp
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:(("python",pid=35481,fd=3))

```

Part 3 UFW Policy Implementation

Remote safety: If you are connected through SSH, keep the existing session open and use the VM console for recovery.

1. Reset UFW only if the instructor confirms the VM contains no required existing rules.
2. Set default deny for incoming traffic and default allow for outgoing traffic.
3. Allow SSH from the private laboratory subnet before enabling UFW.
4. Allow the temporary web service only from the client address.
5. Add an explicit deny rule for Telnet and display numbered rules.

```

sudo ufw status verbose
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp comment 'CORE102 SSH'
sudo ufw allow from 192.168.56.20 to any port 8080 proto tcp comment 'CORE102 test web'
sudo ufw deny 23/tcp comment 'Block Telnet'
sudo ufw enable
sudo ufw status numbered
sudo ss -ltnp

```

```
(obotu@kali)-[~]
$ sudo ufw default deny outgoing
Default outgoing policy changed to 'deny'
(be sure to update your rules accordingly)

(obotu@kali)-[~]
$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)

(obotu@kali)-[~]
$ sudo ufw allow from 192.168.56.0/24 to any port 22 proto tcp comment 'CORE102 SSH'
Rules updated

(obotu@kali)-[~]
$ sudo ufw allow from 192.168.56.0/24 to any port 8080 proto tcp comment 'CORE102 test web'
Rules updated

(obotu@kali)-[~]
$ sudo ufw deny 23/tcp comment 'Block Telnet'
Rules updated

(obotu@kali)-[~]
$ sudo ufw deny 23/tcp comment 'Block Telnet'
Rules updated
Rules updated (v6)

(obotu@kali)-[~]
$ sudo ufw enable
Firewall is active and enabled on system startup

(obotu@kali)-[~]
$ sudo ufw status numbered
Status: active

To: -- Action From
-- --
[ 1] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 2] 8080/tcp ALLOW IN 192.168.56.0/24 # CORE102 test web
[ 3] 23/tcp DENY IN Anywhere # Block Telnet
[ 4] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

(obotu@kali)-[~]
$ sudo ss -ltnp
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
```


Test	Expected	Observed	Matching rule or explanation
HTTP 8080 from other client	Denied	Connection timed out / Refused	Blocked by default UFW deny policy (or lack of rule for other IPs).
Telnet 23	Denied	Connection timed out / Refused	Explicitly denied or dropped by rule matching port 23.

```

File  Machine  View  Input  Devices  Help

vboxuser@UbuntuServer: ~
Last login: Tue Sep 22 11:13:30 2026 from 192.168.1.96
vboxuser@UbuntuServer:~$ ip -br addr
to UNKNOWN 227.0.0.1/8 ::1/128
enp0s3      UP 192.168.56.103/24 metric 100
vboxuser@UbuntuServer:~$ ip route
192.168.56.0/24 dev enp0s3 proto kernel scope link src 192.168.56.103 metric 100
vboxuser@UbuntuServer:~$ nc -vz 192.168.56.104 22
Connection to 192.168.56.104 22 port [tcp/ssh] succeeded!
vboxuser@UbuntuServer:~$ nc -vz 192.168.56.104 8080
Connection to 192.168.56.104 8080 port [tcp/http-alt] succeeded!
vboxuser@UbuntuServer:~$ nc -vz -w 3 192.168.56.104 23
nc: connect to 192.168.56.104 port 23 (tcp) timed out: Operation now in progr
ss
vboxuser@UbuntuServer:~$ curl http://192.168.56.104:8080/
192.168.56.103 - - [30/Sep/2026 16:39:57] "GET / HTTP/1.1" 200 -
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href=".bash_logout">.bash_logout</a></li>
<li><a href=".bashrc">.bashrc</a></li>
<li><a href=".bashrc.original">.bashrc.original</a></li>
<li><a href=".cache/">.cache/</a></li>
<li><a href=".config/">.config/</a></li>
<li><a href=".dmrc">.dmrc</a></li>

```



```
obotu@kali:~$ sudo ufw status numbered
[sudo] password for obotu:
Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 2] 23/tcp DENY IN Anywhere # Block Telnet
[ 3] 8080/tcp ALLOW IN 192.168.56.103 # CORE102 test web
[ 4] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

obotu@kali:~$ sudo journalctl -k --since '10 minutes ago' | grep -i ufw | tail -30

obotu@kali:~$ sudo iptables -L -n -v --line-numbers
Chain INPUT (policy DROP 1 packets, 635 bytes)
num pkts bytes target prot opt in out source destination
1 1848 178K ufw-before-logging-input all -- * 0.0.0.0/0 0.0.0.0/0
2 1848 178K ufw-before-input all -- * 0.0.0.0/0 0.0.0.0/0
3 109 8771 ufw-after-input all -- * 0.0.0.0/0 0.0.0.0/0
4 1 635 ufw-after-logging-input all -- * 0.0.0.0/0 0.0.0.0/0
5 1 635 ufw-reject-input all -- * 0.0.0.0/0 0.0.0.0/0
6 1 635 ufw-track-input all -- * 0.0.0.0/0 0.0.0.0/0

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num pkts bytes target prot opt in out source destination
1 0 0 ufw-before-logging-forward all -- * 0.0.0.0/0 0.0.0.0/0
2 0 0 ufw-before-forward all -- * 0.0.0.0/0 0.0.0.0/0
3 0 0 ufw-after-forward all -- * 0.0.0.0/0 0.0.0.0/0
4 0 0 ufw-after-logging-forward all -- * 0.0.0.0/0 0.0.0.0/0
5 0 0 ufw-reject-forward all -- * 0.0.0.0/0 0.0.0.0/0
6 0 0 ufw-track-forward all -- * 0.0.0.0/0 0.0.0.0/0

Chain OUTPUT (policy ACCEPT 3 packets, 120 bytes)
num pkts bytes target prot opt in out source destination
1 1743 163K ufw-before-logging-output all -- * 0.0.0.0/0 0.0.0.0/0
2 1743 163K ufw-before-output all -- * 0.0.0.0/0 0.0.0.0/0
3 14 3685 ufw-after-output all -- * 0.0.0.0/0 0.0.0.0/0
4 14 3685 ufw-after-logging-output all -- * 0.0.0.0/0 0.0.0.0/0
5 14 3685 ufw-reject-output all -- * 0.0.0.0/0 0.0.0.0/0
6 14 3685 ufw-track-output all -- * 0.0.0.0/0 0.0.0.0/0
```



```
File Machine View Input Devices Help

obotu@kali:~$ sudo iptables -L -n -v --line-numbers
Chain INPUT (policy DROP 1 packets, 635 bytes)
num pkts bytes target prot opt in out source destination
1 1848 178K ufw-before-logging-input all -- * 0.0.0.0/0 0.0.0.0/0
2 1848 178K ufw-before-input all -- * 0.0.0.0/0 0.0.0.0/0
3 109 8771 ufw-after-input all -- * 0.0.0.0/0 0.0.0.0/0
4 1 635 ufw-after-logging-input all -- * 0.0.0.0/0 0.0.0.0/0
5 1 635 ufw-reject-input all -- * 0.0.0.0/0 0.0.0.0/0
6 1 635 ufw-track-input all -- * 0.0.0.0/0 0.0.0.0/0

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num pkts bytes target prot opt in out source destination
1 0 0 ufw-before-logging-forward all -- * 0.0.0.0/0 0.0.0.0/0
2 0 0 ufw-before-forward all -- * 0.0.0.0/0 0.0.0.0/0
3 0 0 ufw-after-forward all -- * 0.0.0.0/0 0.0.0.0/0
4 0 0 ufw-after-logging-forward all -- * 0.0.0.0/0 0.0.0.0/0
5 0 0 ufw-reject-forward all -- * 0.0.0.0/0 0.0.0.0/0
6 0 0 ufw-track-forward all -- * 0.0.0.0/0 0.0.0.0/0

Chain OUTPUT (policy ACCEPT 3 packets, 120 bytes)
num pkts bytes target prot opt in out source destination
1 1743 163K ufw-before-logging-output all -- * 0.0.0.0/0 0.0.0.0/0
2 1743 163K ufw-before-output all -- * 0.0.0.0/0 0.0.0.0/0
3 14 3685 ufw-after-output all -- * 0.0.0.0/0 0.0.0.0/0
4 14 3685 ufw-after-logging-output all -- * 0.0.0.0/0 0.0.0.0/0
5 14 3685 ufw-reject-output all -- * 0.0.0.0/0 0.0.0.0/0
6 14 3685 ufw-track-output all -- * 0.0.0.0/0 0.0.0.0/0

Chain ufw-after-forward (1 references)
num pkts bytes target prot opt in out source destination
1 60 4680 ufw-skip-to-policy-input udp -- * 0.0.0.0/0 0.0.0.0/0 udp dpt:137
2 0 0 ufw-skip-to-policy-input udp -- * 0.0.0.0/0 0.0.0.0/0 udp dpt:138
3 0 0 ufw-skip-to-policy-input tcp -- * 0.0.0.0/0 0.0.0.0/0 tcp dpt:139
4 0 0 ufw-skip-to-policy-input tcp -- * 0.0.0.0/0 0.0.0.0/0 tcp dpt:145
5 0 0 ufw-skip-to-policy-input udp -- * 0.0.0.0/0 0.0.0.0/0 udp dpt:157
6 0 0 ufw-skip-to-policy-input udp -- * 0.0.0.0/0 0.0.0.0/0 udp dpt:168
7 48 3456 ufw-skip-to-policy-input all -- * 0.0.0.0/0 0.0.0.0/0 ADORTYPE match dst-type BROADCAST

Chain ufw-after-logging-forward (1 references)
num pkts bytes target prot opt in out source destination
1 0 0 LOG all -- * 0.0.0.0/0 0.0.0.0/0 limit: avg 3/min burst 10 LOG flags 0 level 4 prefix "[UFW BLOCK]"

Chain ufw-after-logging-input (1 references)
num pkts bytes target prot opt in out source destination
1 0 0 LOG all -- * 0.0.0.0/0 0.0.0.0/0 limit: avg 3/min burst 10 LOG flags 0 level 4 prefix "[UFW BLOCK]"

Chain ufw-after-logging-output (1 references)
num pkts bytes target prot opt in out source destination

Chain ufw-after-output (1 references)
num pkts bytes target prot opt in out source destination

Chain ufw-before-forward (1 references)
num pkts bytes target prot opt in out source destination
1 0 0 ACCEPT all -- * 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLISHED
2 0 0 ACCEPT icmp -- * 0.0.0.0/0 0.0.0.0/0 icmptype 3
3 0 0 ACCEPT icmp -- * 0.0.0.0/0 0.0.0.0/0 icmptype 11
4 0 0 ACCEPT icmp -- * 0.0.0.0/0 0.0.0.0/0 icmptype 12
5 0 0 ACCEPT icmp -- * 0.0.0.0/0 0.0.0.0/0 icmptype 8
6 0 0 ufw-user-forward all -- * 0.0.0.0/0 0.0.0.0/0

Chain ufw-before-input (1 references)
num pkts bytes target prot opt in out source destination
1 4 288 ACCEPT all -- lo * 0.0.0.0/0 0.0.0.0/0 ctstate INVALID
2 1725 169K ACCEPT all -- * 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLISHED
3 0 0 ufw-logging-deny all -- * 0.0.0.0/0 0.0.0.0/0 ctstate INVALID
4 0 0 DROP all -- * 0.0.0.0/0 0.0.0.0/0 ctstate INVALID
5 0 0 ACCEPT icmp -- * 0.0.0.0/0 0.0.0.0/0 icmptype 3
6 0 0 ACCEPT icmp -- * 0.0.0.0/0 0.0.0.0/0 icmptype 11
7 0 0 ACCEPT icmp -- * 0.0.0.0/0 0.0.0.0/0 icmptype 12
```



The top screenshot shows the following iptables rules:

```

4 0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0 ctstate INVALID
5 0 0 ACCEPT icmp -- * * 0.0.0.0/0 0.0.0.0/0 icmptype 3
6 0 0 ACCEPT icmp -- * * 0.0.0.0/0 0.0.0.0/0 icmptype 11
7 0 0 ACCEPT icmp -- * * 0.0.0.0/0 0.0.0.0/0 icmptype 12
8 1 84 ACCEPT icmp -- * * 0.0.0.0/0 0.0.0.0/0 icmptype 8
9 0 0 ACCEPT udp -- * * 0.0.0.0/0 0.0.0.0/0 udp spt:67 dpt:68
10 118 9311 ufw-not-local all -- * * 0.0.0.0/0 0.0.0.0/0
11 0 0 ACCEPT udp -- * * 0.0.0.0/0 224.0.0.251 udp dpt:5353
12 0 0 ACCEPT udp -- * * 0.0.0.0/0 239.255.255.250 udp dpt:1900
13 118 9311 ufw-user-input all -- * * 0.0.0.0/0 0.0.0.0/0

Chain ufw-before-logging-forward (1 references)
num pkts bytes target prot opt in out source destination
Chain ufw-before-logging-input (1 references)
num pkts bytes target prot opt in out source destination
Chain ufw-before-logging-output (1 references)
num pkts bytes target prot opt in out source destination
Chain ufw-before-output (1 references)
num pkts bytes target prot opt in out source destination
1 4 280 ACCEPT all -- * lo 0.0.0.0/0 0.0.0.0/0
2 1725 159K ACCEPT all -- * 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLISHED
3 14 3685 ufw-user-output all -- * * 0.0.0.0/0 0.0.0.0/0
Chain ufw-logging-allow (0 references)
num pkts bytes target prot opt in out source destination
1 0 0 LOG all -- * * 0.0.0.0/0 0.0.0.0/0 limit: avg 3/min burst 10 LOG flags 0 level 4 prefix "[UFW ALLOW] "
Chain ufw-logging-deny (2 references)
num pkts bytes target prot opt in out source destination
1 0 0 RETURN all -- * * 0.0.0.0/0 0.0.0.0/0 ctstate INVALID limit: avg 3/min burst 10
2 0 0 LOG all -- * * 0.0.0.0/0 0.0.0.0/0 limit: avg 3/min burst 10 LOG flags 0 level 4 prefix "[UFW BLOCK] "
Chain ufw-not-local (1 references)
num pkts bytes target prot opt in out source destination
1 9 540 RETURN all -- * * 0.0.0.0/0 0.0.0.0/0 ADDRTYPE match dst-type LOCAL
2 1 635 RETURN all -- * * 0.0.0.0/0 0.0.0.0/0 ADDRTYPE match dst-type MULTICAST
3 108 8136 RETURN all -- * * 0.0.0.0/0 0.0.0.0/0 ADDRTYPE match dst-type BROADCAST
4 0 0 ufw-logging-deny all -- * * 0.0.0.0/0 0.0.0.0/0 limit: avg 3/min burst 10
5 0 0 DROP all -- * * 0.0.0.0/0 0.0.0.0/0
Chain ufw-reject-forward (1 references)
num pkts bytes target prot opt in out source destination
Chain ufw-reject-input (1 references)
num pkts bytes target prot opt in out source destination

```

The bottom screenshot shows the following iptables rules:

```

Chain ufw-skip-to-policy-input (7 references)
num pkts bytes target prot opt in out source destination
1 108 8136 DROP all -- * * 0.0.0.0/0 0.0.0.0/0
Chain ufw-skip-to-policy-output (0 references)
num pkts bytes target prot opt in out source destination
1 0 0 ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0
Chain ufw-track-forward (1 references)
num pkts bytes target prot opt in out source destination
Chain ufw-track-input (1 references)
num pkts bytes target prot opt in out source destination
Chain ufw-track-output (1 references)
num pkts bytes target prot opt in out source destination
1 1 60 ACCEPT tcp -- * * 0.0.0.0/0 0.0.0.0/0 ctstate NEW
2 10 3420 ACCEPT udp -- * * 0.0.0.0/0 0.0.0.0/0 ctstate NEW
Chain ufw-user-forward (1 references)
num pkts bytes target prot opt in out source destination
Chain ufw-user-input (1 references)
num pkts bytes target prot opt in out source destination
1 1 60 ACCEPT tcp -- * * 192.168.56.8/24 0.0.0.0/0 tcp dpt:22
2 3 180 DROP tcp -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:23
3 3 180 ACCEPT tcp -- * * 192.168.56.103 0.0.0.0/0 tcp dpt:8080
Chain ufw-user-limit (0 references)
num pkts bytes target prot opt in out source destination
1 0 0 LOG all -- * * 0.0.0.0/0 0.0.0.0/0 limit: avg 3/min burst 5 LOG flags 0 level 4 prefix "[UFW LIMIT BLOCK] "
2 0 0 REJECT all -- * * 0.0.0.0/0 0.0.0.0/0 reject-with icmp-port-unreachable
Chain ufw-user-limit-accept (0 references)
num pkts bytes target prot opt in out source destination
1 0 0 ACCEPT all -- * * 0.0.0.0/0 0.0.0.0/0
Chain ufw-user-logging-forward (0 references)
num pkts bytes target prot opt in out source destination
Chain ufw-user-logging-input (0 references)
num pkts bytes target prot opt in out source destination
Chain ufw-user-logging-output (0 references)
num pkts bytes target prot opt in out source destination
Chain ufw-user-output (1 references)
num pkts bytes target prot opt in out source destination

```

Part 5 Safe iptables Chain Exercise

This exercise creates a custom chain but does not attach it to INPUT, OUTPUT or FORWARD. It demonstrates rule structure without changing live traffic flow.

```

sudo iptables -N CORE102_LAB
sudo iptables -A CORE102_LAB -p tcp --dport 23 -j DROP
sudo iptables -A CORE102_LAB -p tcp --dport 22 -j ACCEPT
sudo iptables -L CORE102_LAB -n -v --line-numbers
sudo iptables -S CORE102_LAB
sudo iptables -F CORE102_LAB

```

```
sudo iptables -X CORE102_LAB
sudo iptables -L CORE102_LAB 2>&1 || echo "Custom chain removed"
```

1. Label the tool, action, protocol match, destination-port match and target in each rule.

Tool: iptables

Action: -A (Append rule)

Protocol Match: -p tcp

Destination-Port Match: --dport 23 / --dport 22

Target: -j DROP / -j ACCEPT

2. Explain why the custom chain has no effect until another chain jumps to it.

The Linux kernel packet filtering framework evaluates packets by walking through standard built-in chains (INPUT, OUTPUT, FORWARD). Unless a rule in a built-in chain explicitly redirects or jumps (-j) traffic to a user-defined chain using -j CORE102_LAB, the kernel completely ignores the rules inside the custom chain, keeping live traffic entirely unaffected.

3. Confirm that the chain is removed at the end of the exercise.

The command sequence flushes (-F) and deletes (-X) the custom chain. When `sudo iptables -L CORE102_LAB` runs at the end, it returns an error because the chain no longer exists, triggering the `|| echo "Custom chain removed"` fallback message as required.

```
obotu@kali:~$ sudo iptables -N CORE102_LAB
obotu@kali:~$ sudo iptables -A CORE102_LAB -p tcp --dport 23 -j DROP
obotu@kali:~$ sudo iptables -A CORE102_LAB -p tcp --dport 22 -j ACCEPT
obotu@kali:~$ sudo iptables -L CORE102_LAB -n -v --line-numbers
Chain CORE102_LAB (0 references)
num  pkts bytes target    prot opt in     out     source    destination
1     0     0 DROP      tcp  --  *      *       0.0.0.0/0 0.0.0.0/0    tcp dpt:23
2     0     0 ACCEPT    tcp  --  *      *       0.0.0.0/0 0.0.0.0/0    tcp dpt:22
obotu@kali:~$ sudo iptables -F CORE102_LAB
obotu@kali:~$ sudo iptables -X CORE102_LAB
iptables: No chain/target/match by that name.
Custom chain removed
obotu@kali:~$
```

Part 6 SSH Key Authentication

1. On the client, create a dedicated Ed25519 key pair with a strong passphrase.
2. Display and record the public-key fingerprint. Never display or submit the private key contents.
3. Copy the public key to the server and test a new SSH session using that key.
4. Confirm the server sees public-key authentication in its logs.

```
# Client
mkdir -p ~/.ssh
chmod 700 ~/.ssh
ssh-keygen -t ed25519 -a 64 -f ~/.ssh/core102_ed25519 -C "CORE102 authorised lab"
ssh-keygen -lf ~/.ssh/core102_ed25519.pub
```

```
ssh-copy-id -i ~/.ssh/core102_ed25519.pub student@192.168.56.10
ssh -i ~/.ssh/core102_ed25519 student@192.168.56.10

# Server
sudo journalctl -u ssh --since '10 minutes ago' --no-pager | tail -40
stat -c '%a %n' ~/.ssh ~/.ssh/authorized_keys
```

```
File Machine View Input Devices Help
obotu@kali: ~
Session Actions Edit View Help
</html>
vboxuser@UbuntuServer:~$ mkdir -p ~/.ssh
vboxuser@UbuntuServer:~$ chmod 700 ~/.ssh
vboxuser@UbuntuServer:~$ ssh-keygen -t ed25519 -a 64 -f ~/.ssh/core102_ed25519 -C "CORE102 authorised lab"
Generating public/private ed25519 key pair.
Enter passphrase for "/home/vboxuser/.ssh/core102_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/vboxuser/.ssh/core102_ed25519
Your public key has been saved in /home/vboxuser/.ssh/core102_ed25519.pub
The key fingerprint is:
SHA256:piEBN8+3NxFoZJ5a2rcmGR/RWoLWwGS43/hXoq9sw3w CORE102 authorised lab
The key's randomart image is:
+--[ED25519 256]--+
|      .+      .+      |
|    .++ .+ .+      |
|  ..+O=...      |
| . .-.O.O.      |
| ..= .S=O      |
|  .++=...      |
|  .+ =      |
|   X E      |
|  .O.      |
+--[SHA256]-----+
vboxuser@UbuntuServer:~$ ssh-keygen -lf ~/.ssh/core102_ed25519.pub
256 SHA256:piEBN8+3NxFoZJ5a2rcmGR/RWoLWwGS43/hXoq9sw3w CORE102 authorised lab (ED25519)
vboxuser@UbuntuServer:~$ ssh-copy-id -i ~/.ssh/core102_ed25519.pub obotu@192.168.56.104
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/vboxuser/.ssh/core102_ed25519.pub"
The authenticity of host '192.168.56.104 (192.168.56.104)' can't be established.
ED25519 key fingerprint is: SHA256:ZaQmxAS584c/LTHlfESQhPmKAzVSp9ALxohhASng/Kc
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that
are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is
to install the new keys
obotu@192.168.56.104's password:
Number of key(s) added: 1
Now try logging into the machine, with: "ssh -i /home/vboxuser/.ssh/core102_ed25519 'obotu@192.168.56.104'"
and check to make sure that only the key(s) you wanted were added.
vboxuser@UbuntuServer:~$ ssh -i ~/.ssh/core102_ed25519 obotu@192.168.56.104
Enter passphrase for key '/home/vboxuser/.ssh/core102_ed25519':
Linux kali 6.19.14-kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1kali1 (2026-05-05) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
obotu@kali:~$
obotu@kali:~$ ssh-copy-id -i ~/.ssh/core102_ed25519.pub obotu@192.168.56.104
/usr/bin/ssh-copy-id: ERROR: failed to open ID file '/home/obotu/.ssh/core102_ed25519.pub': No such file
obotu@kali:~$ sudo journalctl -u ssh --since '10 minutes ago' --no-pager | tail -40
[sudo] password for obotu:
Sep 30 17:00:47 kali sshd-session[36156]: Connection closed by authenticating user obotu 192.168.56.103 port 33988 [preauth]
Sep 30 17:00:47 kali sshd-session[36254]: Connection closed by authenticating user obotu 192.168.56.103 port 47598 [preauth]
Sep 30 17:07:07 kali sshd-session[36257]: Accepted password for obotu from 192.168.56.103 port 47604 ssh2
Sep 30 17:07:07 kali sshd-session[36257]: pam_unix(sshd:session): session opened for user obotu(uid=1000) by obotu(uid=0)
Sep 30 17:07:07 kali sshd-session[36257]: pam_unix(sshd:session): session closed for user obotu
Sep 30 17:08:02 kali sshd-session[36843]: Accepted publickey for obotu from 192.168.56.103 port 55652 ssh2: ED25519 SHA256:piEBN8+3NxFoZJ5a2rcmGR/RWoLWwGS43/hXoq9sw3w
Sep 30 17:08:02 kali sshd-session[36843]: pam_unix(sshd:session): session opened for user obotu(uid=1000) by obotu(uid=0)
obotu@kali:~$ stat -c '%a %n' ~/.ssh ~/.ssh/authorized_keys
800 /home/obotu/.ssh
800 /home/obotu/.ssh/authorized_keys
obotu@kali:~$
```

Part 7 SSH Server Hardening

Gate: Do not disable password authentication until key-based login succeeds in a separate session.

1. Back up the current SSH server configuration.
2. Create a small configuration drop-in rather than rewriting the full main file.
3. Validate the configuration with `sshd -t`. Do not reload if validation reports any error.
4. Keep the original session open, reload SSH and test a second key-based session.

```
# Server
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.core102.bak
sudo nano /etc/ssh/sshd_config.d/90-core102-hardening.conf
```

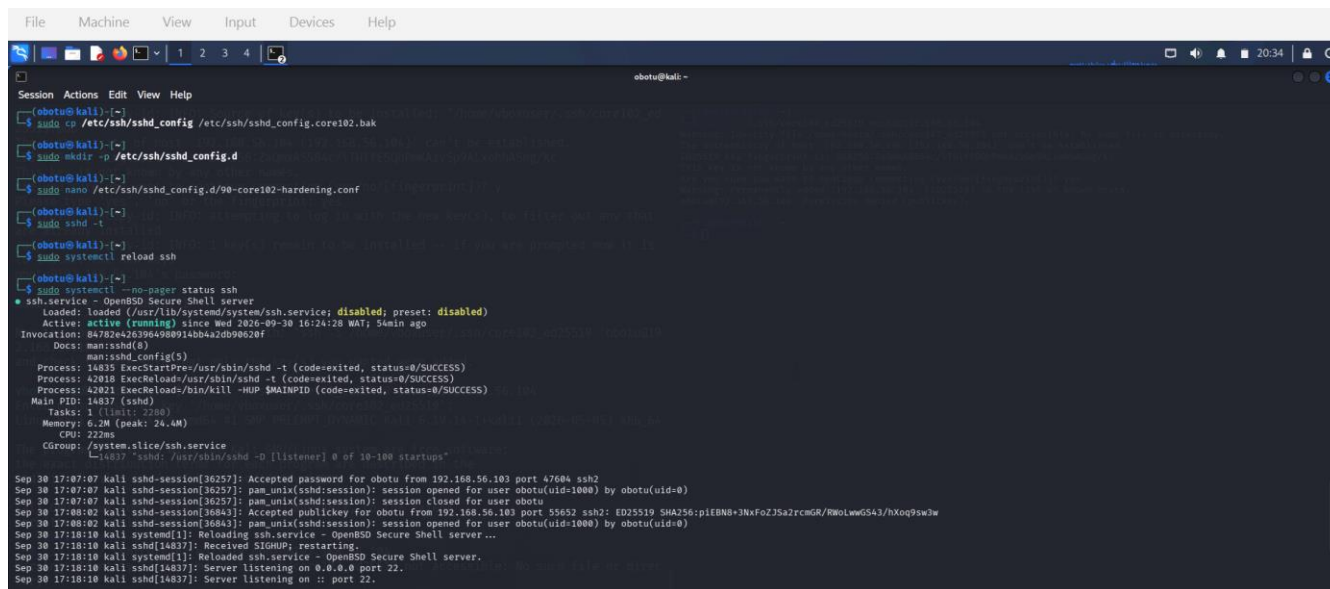
Enter the following settings in the drop-in file:

```
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no

sudo sshd -t
sudo systemctl reload ssh
sudo systemctl --no-pager status ssh

# Client in a second terminal
ssh -i ~/.ssh/core102_ed25519 student@192.168.56.10
```

If key authentication fails, use the VM console to restore `/etc/ssh/sshd_config.core102.bak`, remove the drop-in file, run `sudo sshd -t` and reload SSH.



```
File Machine View Input Devices Help
obotu@kali: ~
Session Actions Edit View Help
(obotu@kali)~$ sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.core102.bak
(obotu@kali)~$ sudo mkdir -p /etc/ssh/sshd_config.d
(obotu@kali)~$ sudo nano /etc/ssh/sshd_config.d/90-core102-hardening.conf
(obotu@kali)~$ sudo sshd -t
(obotu@kali)~$ sudo systemctl reload ssh
(obotu@kali)~$ sudo systemctl --no-pager status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: active (running) since Wed 2020-09-30 10:24:28 WAT; 54min ago
   Invocation: 84782e4263964980914b04a2db98020f
   Docs: man:ssh(8)
        man:ssh_config(5)
   Process: 14835 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Process: 42018 ExecReload=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Process: 42021 ExecReload=/bin/kill -HUP $MAINPID (code=exited, status=0/SUCCESS)
   Main PID: 14837 (sshd)
   Tasks: 1 (limit: 2280)
   Memory: 6.2M (peak: 24.4M)
   CPU: 222ms
   CGroup: /system.slice/ssh.service
           └─14837 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Sep 30 17:07:07 kali sshd-session[36257]: Accepted password for obotu from 192.168.56.103 port 47604 ssh2
Sep 30 17:07:07 kali sshd-session[36257]: pam_unix(sshd:session): session opened for user obotu(uid=1000) by obotu(uid=0)
Sep 30 17:07:07 kali sshd-session[36257]: pam_unix(sshd:session): session closed for user obotu
Sep 30 17:08:02 kali sshd-session[36843]: Accepted publickey for obotu from 192.168.56.103 port 55652 ssh2: ED25519 SHA256:plEBN8+3NxFoZJ5a2rcmGR/RW0LwG543/hXoq9swW
Sep 30 17:08:02 kali sshd-session[36843]: pam_unix(sshd:session): session opened for user obotu(uid=1000) by obotu(uid=0)
Sep 30 17:18:10 kali systemd[1]: Reloading ssh.service - OpenBSD Secure Shell server...
Sep 30 17:18:10 kali sshd[14837]: Received SIGHUP: restarting.
Sep 30 17:18:10 kali systemd[1]: Reloaded ssh.service - OpenBSD Secure Shell server.
Sep 30 17:18:10 kali sshd[14837]: Server listening on 0.0.0.0 port 22.
Sep 30 17:18:10 kali sshd[14837]: Server listening on :: port 22.
```

```
vboxuser@UbuntuServer:~$ ssh -i ~/.ssh/core102_ed25519 obotu@192.168.56.104
Enter passphrase for key '/home/vboxuser/.ssh/core102_ed25519':
Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64
```

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in `/usr/share/doc/*/copyright`.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

Part 8 Final Exposure Review and Cleanup

1. Stop the Python HTTP server and prove that port 8080 no longer listens.
2. Remove the temporary UFW port 8080 rule by its current numbered index. Re-list the rules immediately after deletion.
3. Confirm SSH remains accessible from the authorised client.
4. Compare the final socket inventory with the baseline and explain every remaining listening service.
5. Do not delete the client private key until the instructor confirms that evidence review is complete.

```
sudo ss -tulpn
sudo ufw status numbered
# Replace N with the current number of the temporary 8080 rule
sudo ufw delete N
sudo ufw status numbered
sudo ss -ltnp | grep ':8080' || echo "Temporary web service stopped"
```

```
File Machine View Input Devices Help
obotu@kali: ~
Session Actions Edit View Help
~$ sudo ss -ltnp | grep ':8080' || echo "Temporary web service stopped"
[sudo] password for obotu:
LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:((("python3",pid=14187,fd=3))
(obotu@kali)~$ sudo ufw status numbered
Status: active
To Action From
[ 1] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 2] 23/tcp DENY IN Anywhere # Block Telnet
[ 3] 8080/tcp ALLOW IN 192.168.56.103 # CORE102 test web
[ 4] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet
(obotu@kali)~$ sudo ufw delete N # (Replace N with the rule number for 8080)
ERROR: Invalid syntax
Usage: ufw COMMAND
Commands:
enable enables the firewall
disable disables the firewall
default ARG set default policy
logging LEVEL set logging to LEVEL
allow ARGS add allow rule
deny ARGS add deny rule
reject ARGS add reject rule
limit ARGS add limit rule
delete RULE|NUM delete RULE
insert NUM RULE insert RULE at NUM
prepend RULE prepend RULE
route RULE add route RULE
route delete RULE|NUM delete route RULE
route insert NUM RULE insert route RULE at NUM
reload reload firewall
reset reset firewall
status show firewall status
status numbered show firewall status as numbered list of RULES
status verbose show verbose firewall status
show ARG show firewall report
version display version information
Application profile commands:
app list list application profiles
app info PROFILE show information on PROFILE
app update PROFILE update PROFILE
app default ARG set default application policy
(obotu@kali)~$ sudo ufw status numbered
Status: active
To Action From
[ 1] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 2] 23/tcp DENY IN Anywhere # Block Telnet
[ 3] 8080/tcp ALLOW IN 192.168.56.103 # CORE102 test web
[ 4] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet
(obotu@kali)~$ sudo ss -tlnp
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:((("sshd",pid=14837,fd=6))
tcp LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:((("python3",pid=14187,fd=3))
tcp LISTEN 0 128 [*]:22 [*]:* users:((("sshd",pid=14837,fd=7))
(obotu@kali)~$
```

The screenshot shows a Kali Linux terminal window with the following content:

```
obotu@kali:~$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 2] 23/tcp DENY IN Anywhere # Block Telnet
[ 3] 8080/tcp ALLOW IN 192.168.56.103 # CORE102 test web
[ 4] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

obotu@kali:~$ sudo ss -tulnp
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:([sshd,pid=14837,fd=6])
tcp LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:([python3,pid=14187,fd=3])
tcp LISTEN 0 128 0.0.0.0:23 0.0.0.0:* users:([sshd,pid=14837,fd=7])

obotu@kali:~$ sudo ufw delete 3
Deleting:
allow from 192.168.56.103 to any port 8080 proto tcp comment 'CORE102 test web'
Proceed with operation (y/n)? y
Rule deleted

obotu@kali:~$ sudo ufw status numbered
Status: active

To Action From
--
[ 1] 22/tcp ALLOW IN 192.168.56.0/24 # CORE102 SSH
[ 2] 23/tcp DENY IN Anywhere # Block Telnet
[ 3] 23/tcp (v6) DENY IN Anywhere (v6) # Block Telnet

obotu@kali:~$ sudo ss -tulnp
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:* users:([sshd,pid=14837,fd=6])
tcp LISTEN 0 5 0.0.0.0:8080 0.0.0.0:* users:([python3,pid=14187,fd=3])
tcp LISTEN 0 128 0.0.0.0:23 0.0.0.0:* users:([sshd,pid=14837,fd=7])
```

Submission Package

Submit one PDF report and one ZIP archive. Use your registration number in both filenames.

- REGNO_CORE102_Lab4_Report.pdf

- REGNO_CORE102_Lab4_Files.zip containing only public configuration evidence and the SSH public key file

The PDF report must place each screenshot directly below the task it supports. Every screenshot must show the command and its output. Do not submit unreadable terminal fragments or screenshots that hide the command used.

Reflection Questions and answers:

1. Why must SSH access be allowed before a default-deny firewall is enabled?

- **Preventing Lockout:** If a firewall's default policy is set to block incoming traffic (deny-all) before explicitly creating an allow rule for SSH, all remote communication streams to the server are instantly severed.
- **Maintaining Administrative Control:** Because remote management depends entirely on active network sockets, locking out management ports forces administrators to rely on local console access (such as a physical hypervisor or VM console window) to recover connectivity.

2. What is the difference between an open port and an authorised service?

- **Open Port:** A listening socket on the network stack (e.g., as reported by ss or netstat) indicates that an operating system process is bound to a network port and is actively waiting for incoming packets.
- **Authorised Service:** An authorised service encompasses both the running application and the strict access-control policies (such as firewall rules, authentication mechanisms, and source-IP restrictions) that legally and securely permit specific clients to interact with that port. A port can technically be open while remaining unauthorised or blocked to unauthorized networks.

3. Why does the server need the public key while the client must protect the private key?

- **Server-Side Public Key (authorized_keys):** The server uses the stored public key solely to mathematically challenge and verify cryptographic signatures presented by connecting clients during public-key authentication, without ever needing or storing the private key.
- **Client-Side Private Key (core102_ed25519):** The private key is the sole secret credential used to sign authentication challenges. If a private key is compromised, any attacker possessing it can impersonate the authorized user and bypass authentication entirely, making its strict confidentiality mandatory.

4. Which rollback step would recover access after an invalid SSH configuration?

- **Console Restoration:** If an invalid configuration locks out administrators during sshd reloading, access is restored via the VM hypervisor console.
- **Configuration Reversion:** From the console, the backup file is restored using the command `sudo cp /etc/ssh/sshd_config.core102.bak /etc/ssh/sshd_config`, followed by removing faulty drop-in files, validating syntax with `sudo sshd -t`, and restarting the SSH daemon with `sudo systemctl restart ssh`.